

1. Propósito, alcance y estado del diseño

Esta memoria propone una evolución del backend de Alerta Satipo hacia puertos y adaptadores. La versión vigente es un frontend estático que consulta Supabase directamente mediante JavaScript. No existen todavía un servidor Express, contratos de repositorios ni casos de uso independientes. Las láminas describen el objetivo, no certifican su implementación.

El alcance propuesto es el panel administrativo: consulta de las 300 lecturas, revisión de casos y actualización del mantenimiento demostrativo. Se conservan Supabase Auth y PostgreSQL. Los nodos siguen siendo seis lotes virtuales de 50 lecturas; las etiquetas positivas no se convierten en incendios confirmados.

2. Decisión arquitectónica y responsabilidades

Decisión ADR-001: introducir un servicio de aplicación en Node.js y Express, con dependencias dirigidas hacia contratos propios. Motivación: extraer la coordinación de operaciones del navegador, centralizar autorización y probar reglas sin red. Coste: nuevo despliegue, configuración de CORS y TLS, latencia adicional, gestión de errores y mantenimiento operativo. Para un prototipo pequeño, Supabase directo sigue siendo una solución válida; la evolución se justifica por requisitos académicos y de mantenibilidad, no por una supuesta incapacidad de Supabase.

Dominio: objetos y reglas expresados en JavaScript puro. Aplicación: casos de uso, autorización y coordinación. Puertos de entrada: operaciones que ofrece el núcleo. Puertos de salida: capacidades externas que necesita. Adaptadores: HTTP, persistencia e identidad Supabase. El punto de composición crea las implementaciones e inyecta las dependencias.

La inversión de dependencias ocurre porque el repositorio Supabase implementa un contrato definido por el núcleo. Que el caso de uso invoque al repositorio durante la ejecución no implica que su código dependa de la clase Supabase. Las dos láminas separan deliberadamente estructura y ejecución.

3. Contratos propuestos y reglas verificables

Puerto de entrada: `actualizarMantenimiento({actor, nodeId, expectedState, nextState})` devuelve una tarea actualizada o un error tipado. actor debe provenir de una identidad verificada, no de un campo enviado libremente por el navegador.

Puerto de persistencia: `obtener(nodeId)` devuelve Tarea o ausencia; `guardarSiCoincide(nodeId, expectedState, nextState)` devuelve actualizado o conflicto, y distingue errores de infraestructura. Puerto de identidad: `verificarSesion(token)` y `esAdministrador(userId)`. El adaptador HTTP no conoce tablas; el caso de uso no conoce códigos HTTP; el adaptador de persistencia traduce filas y errores del proveedor.

Invariantes existentes: state pertenece a Pendiente, En progreso o Completada; node_id identifica un nodo existente; una tarea se asocia a un solo nodo. Las restricciones actuales permiten cambiar entre cualquiera de esos estados. No se debe afirmar que hay una máquina de transiciones estricta o prohibición de reabrir tareas: esas serían reglas nuevas que requieren definición.

Concurrencia: el UPDATE filtra por node_id y state esperado; evita sobrescribir un cambio incompatible. Si devuelve cero filas, se consulta existencia con el mismo contexto autorizado para distinguir ausencia de conflicto. Comparar solo el estado no detecta un cambio $A \rightarrow B \rightarrow A$; para garantizar detección completa se propone una columna version e incremento atómico mediante operación SQL/RPC. Esa versión no está implementada.

Atomicidad: actualizar una tarea es una operación SQL. Si se agrega una auditoría obligatoria de mantenimiento, el cambio y el evento deben confirmarse en la misma transacción, mediante función o disparador. Hoy solo los incidentes tienen auditoría; no se afirma que las tablas demo dispongan de ella.

4. Seguridad y persistencia

El adaptador de identidad debe validar la sesión en Supabase y resolver la autorización administrativa en el servidor. No basta decodificar un JWT ni leer sessionStorage. La respuesta nunca debe incluir claves privadas o tokens en registros.

Para persistencia se propone usar clave pública y JWT del usuario en un cliente acotado a la solicitud. Así PostgreSQL conserva RLS como segunda barrera. No utilizar una sesión mutable global compartida entre usuarios. Usar una clave administrativa en las solicitudes ordinarias podría eludir RLS y exigiría controles equivalentes explícitos; no es la opción propuesta.

El traslado al servidor no elimina automáticamente las rutas REST directas de Supabase. Mientras existan los privilegios actuales, un administrador puede seguir modificando state por esa API. Si una regla debe ser imposible de eludir, también debe imponerse en PostgreSQL o restringirse la escritura a una operación controlada. CORS no sustituye esos permisos.

Lecturas históricas: claves dataset_id y source_row preservan la trazabilidad. El puerto de consulta debe incluir filtros y paginación sin devolver objetos del SDK al dominio. La importación de CSV sigue siendo una tarea administrativa separada; no se descarga GitHub en cada consulta del panel.

5. Migración, trazabilidad y aceptación

Paso 1: crear backend/src/domain, application/ports, application/use-cases, adapters/in/http, adapters/out/supabase y bootstrap. Los directorios solo organizan; la independencia se demuestra con importaciones y pruebas.

Paso 2: extraer la actualización de mantenimiento de web/js/operations.js a un caso de uso. Definir los contratos antes de implementar el adaptador. Mantener las reglas SQL de la migración 003.

Paso 3: implementar adaptadores HTTP, identidad y persistencia; trasladar la invocación del panel al nuevo endpoint. Configurar alojamiento del servicio, HTTPS, origen permitido y variables privadas. Firebase Hosting actual publica archivos, no el nuevo proceso Express.

Paso 4: migrar consultas de lecturas desde shared/js/dataset.js y casos desde operations.js; no reescribir todo en una sola entrega. Mantener verificaciones de las 300 filas y los vínculos existentes.

Aceptación: pruebas unitarias del dominio sin SDK; casos de uso con repositorio en memoria; pruebas de contrato que ejecuten la misma suite sobre memoria y Supabase de prueba; pruebas HTTP de 401/403/404/409/422/503; integración que demuestre RLS; dos actualizaciones concurrentes con un mismo estado esperado y destino distinto; prueba de regresión que conserve los 300 registros.

Añadir comprobación automatizada de importaciones: domain no importa Express, Supabase ni módulos de adapters; application no importa implementaciones de infraestructura. La prueba debe fallar si se introduce una dependencia prohibida. No se han ejecutado estas pruebas propuestas porque el servicio todavía no existe.

6. Guion de sustentación y evidencia

Explicación: “Separamos reglas, coordinación e infraestructura. El núcleo define los contratos; HTTP inicia los casos de uso y Supabase implementa persistencia e identidad. La dirección de las dependencias apunta al núcleo, aunque el flujo de ejecución salga hacia la base de datos. Conservamos RLS y restricciones SQL como garantías adicionales. Esta es una propuesta de evolución verificable, no una descripción ficticia del despliegue actual”.

Si preguntan por qué usar hexagonal: permite sustituir adaptadores y probar casos de uso sin red, a cambio de mayor complejidad. Si preguntan si cambiar de base de datos es gratis: no; se reemplaza el adaptador y se migran esquema, políticas y funciones, aunque los casos de uso deberían permanecer estables. Si preguntan qué prueba la arquitectura: los contratos, la composición, las importaciones y las pruebas, no la forma del dibujo.

Fuentes internas: `shared/js/backend.js` (identidad y guard); `web/js/operations.js` (estados y actualización condicional); `shared/js/dataset.js` (lecturas); `supabase/migrations/202609230003_demo_operations.sql` (relaciones y RLS); `tests/database.test.mjs` y `tests/panel_operations.py` (verificación actual). Estos archivos respaldan el punto de partida; no demuestran la implementación del diseño futuro.